

UNIVERSIDADE FEDERAL DE ITAJUBÁ

CAMPUS ITAJUBÁ

GABRIEL PRANGE VILELA - 2024003709

PAULO VINÍCIUS GARCIA BARBOSA RODRIGUES - 2024004081

ROBERTO RIVOLI GOMES JÚNIOR - 2024001811

ECOM06A - COMPILADORES

Analizador Léxico e Sintático da linguagem:

“buteCo”

UNIFEI – Itajubá

Mai de 2026

SUMÁRIO

1 Introdução.....	5
2 Metodologia.....	5
2.1 Modelagem Lexical e Definição do vocabulário.....	6
2.2 Formalização por expressões regulares.....	9
2.3 Regras de produção.....	10
2.4 Transição de Estados e Autômatos finitos.....	12
3 Conclusão.....	16

1 Introdução

Este trabalho visa projetar as especificações léxicas e sintáticas da linguagem imperativa "ButeCo", que adota a "filosofia de bar" em seu vocabulário, mapeando estruturas de C sem perda de rigor.

O projeto inclui a formalização da linguagem por expressões regulares (extração de tokens), conversão em autômatos finitos (ADFs e AFNs) e a estruturação de regras de produção sintática, demonstrando a aplicação da teoria de linguagens formais na criação de um compilador funcional e temático.

2 Metodologia

O desenvolvimento das especificações do analisador léxico e sintático da linguagem passou por quatro etapas metodológicas, nas quais cada fase procurou garantir que a temática da linguagem fosse sustentada por um padrão técnico necessário para o processamento computacional.

Dentro dessa estrutura, a separação entre as Expressões Regulares e as Regras de Produção foi fundamental para otimizar a arquitetura do compilador:

- Camada Léxica (Expressões Regulares): Nesta etapa, o foco foi o reconhecimento de padrões lineares. Utilizamos as Regex para identificar "tokens" (palavras-chave, identificadores e símbolos) e descartar elementos irrelevantes, como espaços e comentários. Por serem baseadas em autômatos finitos, as Regex garantem a eficiência necessária para essa varredura inicial de caracteres.
- Camada Sintática (Regras de Produção): Para validar a estrutura hierárquica e a gramática da linguagem, aplicamos regras de produção em EBNF. Essa escolha justifica-se pela limitação teórica das Expressões Regulares, que não conseguem processar estruturas recursivas ou aninhadas, como blocos de código e precedência de operadores.

Essa divisão facilita a manutenção do sistema e garante melhor interpretação das instruções pelo compilador.

2.1 Modelagem Lexical e Definição do vocabulário

O desenvolvimento desta fase consistiu em levantar os requisitos da linguagem com familiaridades das linguagens C e Python com uma temática única. Esta fase consistiu na abstração de conceitos de programação imperativa para um vocabulário de “bar”, garantindo que os tokens possuísem semântica clara e uma estrutura formal definida. Além da tipagem, mapearam-se palavras reservadas para o controle de fluxo e operadores específicos. Entre as especificidades definidas, destacam-se as presentes na tabela 1.

Especificações da linguagem
Semelhante à linguagem C e Python
Delimitação dos blocos é feita por abre_buteco e fecha_buteco
Variáveis (identificadores) devem obrigatoriamente iniciar com uma letra, podendo ser seguidas por números e underscore (_).
O fim de toda instrução é obrigatoriamente sinalizado pela palavra desce no lugar do tradicional ponto e vírgula.
Diferencia explicitamente a atribuição (anota) da comparação relacional de igualdade (bate_com), prevenindo erros lógicos de desvio condicional.
Possui o operador semântico exclusivo racha (Atribuição Distributiva), que permite dividir o resultado de uma expressão igualmente entre múltiplas variáveis.
Comentários são apenas de linha, sendo iniciados pelo símbolo \$, fazendo com que o analisador léxico ignore o restante do texto até a quebra de linha.
A entrada e saída de dados são tratadas como comandos nativos através de pede_pro_garcom() para leitura e chama_truco() para impressão no terminal.
Introdução do modificador de tipo pre_pago. Variáveis declaradas com este modificador possuem proteção semântica contra valores negativos. Se uma subtração resultar em valor negativo, o compilador força o armazenamento de 0, prevenindo lógicas inconsistentes.
Diferente do C, variáveis declaradas sem atribuição recebem implicitamente o valor 0 (copo vazio), prevenindo o uso de lixo de memória residual.
A vírgula (,) atua exclusivamente como separador de casas decimais em números reais; E o ponto e vírgula (;) atua como separador de elementos em listas e declarações múltiplas.
Uso de ta_pago e pendurado como palavras reservadas para valores booleanos. (respectivamente true/1 e false/0).
Permite a declaração e atribuição de múltiplas variáveis em sequência, com a restrição obrigatória de que a cadeia de atribuições seja sempre finalizada por um número ou expressão aritmética.

Tabela 1 - Especificações da linguagem

Tipo	Referência	buteCo	Analizador Léxico	Token Referência	Token buteCo
Tipos de dados suportados	int	dose	Palavra Reservada	T_INT	T_DOSE
	float	litrao	Palavra Reservada	T_FLOAT	T_LITRAO
	char	petisco	Palavra Reservada	T_CHAR	T_PETISCO
Variável	letra	letra	Literais de Strings	T_ID	T_ID
	número	número	Literais Numéricos	T_NUM	T_NUM
Operadores relacionais	=	anota	Operador de Atribuição	T_ATR	T_ANOTA
	==	bate_com	Operador Relacional	T_EQ	T_BATECOM
	!=	nao_bate	Operador Relacional	T_NEQ	T_NAOBATE
	>=	>=	Operador Relacional	T_GEQ	T_GEQ
	<=	<=	Operador Relacional	T_LEQ	T_LEQ
	>	>	Operador Relacional	T_GT	T_GT
	<	<	Operador Lógico	T_LT	T_LT
Operadores lógicos	&	e	Operador Lógico	T_AND	T_E
		ou	Operador Lógico	T_OR	T_OU
	~	nao	Operador Lógico	T_NOT	T_NAO
Operadores Aritméticos	+	+	Operador Aritmético	T_PLUS	T_PLUS
	-	-	Operador Aritmético	T_MINUS	T_MINUS
	*	*	Operador Aritmético	T_MUL	T_MUL
	/	/	Operador Aritmético	T_DIV	T_DIV

	mod	mod	Operador Aritmético	T_MOD	T_MOD
Símbolos especiais	.	,	Delimitadores	T_DOT	T_DOT
	,	;	Delimitadores	T_COMMA	T_COMMA
	:	:	Delimitadores	T_COLON	T_COLON
	;	desce	Delimitadores	T_DEL	T_DEL
	'	'	Delimitadores	T_SINGLE_QUOTE	T_SINGLE_QUOTE
	"	"	Delimitadores	T_STR	T_STR
	(	(	Delimitadores	T_LPAREN	T_LPAREN
	)	)	Delimitadores	T_RPAREN	T_RPAREN
	{	abre_o_buteco	Delimitadores	T_LBRACE	T_ABREBUTECO
	}	fecha_o_buteco	Delimitadores	T_RBRACE	T_FECHABUTECO
	#	#	Símbolo Especial	T_POUND	T_POUND
	//	\$\$	Símbolo Especial	T_COMMENT	T_COMMENT
	/* */	\$* *\$	Símbolo Especial	T_BLOCK_COMMENT	T_BLOCK_COMMENT
Palavras reservadas	if	se_ta_pago	Palavra Reservada	T_IF	T_SETAPAGO
	else	se_nao	Palavra Reservada	T_ELSE	T_SENAO
	for	rodada	Palavra Reservada	T_FOR	T_RODADA
	while	enche_o_copo	Palavra Reservada	T_WHILE	T_ENCHE
	switch	cardapio	Palavra Reservada	T_SWITCH	T_CARDAPIO
	case	pedido	Palavra Reservada	T_CASE	T_PEDIDO
	cin	pede_pro_garcom	Palavra Reservada	T_CIN	T_PEDEGARCOM
	cout	chama_truco	Palavra Reservada	T_COUT	T_CHAMATRUCO
	void	ressaca	Palavra Reservada	T_VOID	T_RESSACA
	null	copo_vazio	Palavra Reservada	T_NULL	T_COPOVAZIO
	break	lei_seca	Palavra	T_BREAK	T_LEISECA

			Reservada		
	return	saidera	Palavra Reservada	T_RETURN	T_SAIDERA
	define	apelidar	Palavra Reservada	T_DEFINE	T_APELIDAR
	include	desce_uma	Palavra Reservada	T_INCLUDE	T_DESCEUMA
		racha	Palavra Reservada		T_RACHA
		pre_pago	Palavra Reservada		T_PREPAGO
	TRUE	ta_pago	Palavra Reservada	T_TRUE	T_TAPAGO
	FALSE	ta_pendurado	Palavra Reservada	T_FALSE	T_TAPENDURADO

Tabela 2 - Tabela de tokens

2.2 Formalização por expressões regulares

Nesta etapa, cada elemento do vocabulário da linguagem buteCo foi formalizado matematicamente através de expressões regulares, com o objetivo de garantir a extração precisa de tokens pelo analisador léxico. Foram estabelecidas regras para identificadores (T_ID), que devem iniciar obrigatoriamente com uma letra, podendo ser seguidos por letras, dígitos ou underscores. Para literais numéricos (T_NUM), definiu-se o suporte a sinais opcionais de positivo e negativo, além de casas decimais separadas por vírgula conforme a especificação da linguagem, onde a vírgula atua exclusivamente como separador decimal.

Nome	Expressão
num	[0-9]
letra	[a-zA-Z]
T_NUM	(+ -)? num+ (,num+)?
T_ID	letra (letra num _)*
texto	T_STR .* T_STR
relacional	T_BATECOM T_NAOBATE T_GEQ T_LEQ T_GT T_LT
aritmético	T_PLUS T_MINUS T_MUL T_DIV T_MOD

Tabela 3 - Expressões regulares.

Nome	Finalidade
num	Conjunto de todos os algarismos 0 a 9
letra	Conjunto de todas as letras minúsculas e maiúsculas
T_NUM	Identifica um literal numérico, com suporte opcional a sinal e casas decimais separadas por vírgula
T_ID	Identifica um identificador (variável/função): deve iniciar com letra, seguido por letras, dígitos ou underscores
texto	Identifica uma cadeia de caracteres delimitada por aspas duplas
relacional	Conjunto dos operadores de comparação: igualdade, diferença, maior, menor, etc.
aritmético	Conjunto dos operadores aritméticos: soma, subtração, multiplicação, divisão e módulo

Tabela 4 - Explicação das Expressões regulares.

2.3 Regras de produção

As regras de produção são apresentadas na Tabela 5. Para cada regra, utiliza-se a notação EBNF: colchetes [] indicam elementos opcionais (zero ou uma ocorrência), chaves { } indicam repetição de zero ou mais vezes, e a barra vertical | indica alternativa entre produções. Tokens em caixa alta representam símbolos terminais, enquanto nomes com inicial maiúscula representam símbolos não-terminais (outras regras de produção).

Regras de produção	
Programa	T_ABREBUTECO Bloco T_FECHABUTECO
Bloco	{Comando}
DeclaraVar	[T_PREPAGO] Tipos T_ID [{T_ANOTA T_ID} T_ANOTA ExprAritm] T_DEL
Atribuicao	T_ID T_ANOTA {T_ID T_ANOTA} ExprAritmetica T_DEL
Rachadinha	ListaIDs T_RACHA ExprAritmetica T_DEL
ListaIDs	T_ID { T_COMMA T_ID }
Escuta	T_PEDEGARCOM T_LPAREN T_ID T_RPAREN T_DEL

Truco	T_CHAMATRUCO T_LPAREN (ExprAritmetica T_STR) T_RPAREN T_DEL
Condicional	T_SETAPAGO T_LPAREN Condição T_RPAREN T_ABREBUTECO Bloco T_FECHABUTECO [T_SENAO T_ABREBUTECO Bloco T_FECHABUTECO]
Enquanto	T_ENCHE T_LPAREN Condição T_RPAREN T_ABREBUTECO Bloco T_FECHABUTECO
Para_Rodada	T_RODADA T_LPAREN Atribuicao Condição T_DEL Atribuicao T_RPAREN T_ABREBUTECO Bloco T_FECHABUTECO
Escolha_Cardapio	T_CARDAPIO T_LPAREN T_ID T_RPAREN T_ABREBUTECO { T_PEDIDO Fator T_COLON Bloco T_LEISECA T_DEL } T_FECHABUTECO
Retorno	T_SAIDERA T_LPAREN ExprAritmetica T_RPAREN T_DEL
Condição	Álgebra Booleana T_NAO Álgebra Booleana
ExprRelacional	ExprAritmetica relacional ExprAritmetica
Álgebra Booleana	ExprRelacional { (T_E T_OU) ExprRelacional }
ExprAritmetica	Termo { (T_PLUS T_MINUS) Termo }
Termo	Fator { (T_MUL T_DIV T_MOD) Fator }
Fator	T_NUM T_ID T_LPAREN ExprAritmetica T_RPAREN
Comando	DeclaraVar Atribuicao Rachadinha Escuta Truco Condicional Enquanto Para_Rodada Escolha_Cardapio Retorno
Tipos	T_DOSE T_LITRAO T_PETISCO T_RESSACA

Tabela 5 - Regras de produção

2.4 Transição de Estados e Autômatos finitos

A partir das expressões regulares definidas na seção anterior, cada regra de produção da gramática foi convertida em um autômato finito determinístico (AFD) representado por diagrama de transição de estados. Esses autômatos modelam o fluxo de aceitação do analisador sintático, onde cada estado intermediário consome um token específico e transiciona para o próximo estado até atingir o estado de aceitação final. Os diagramas a seguir ilustram os autômatos correspondentes a cada construção sintática da linguagem buteCo. Em cada diagrama, o estado inicial é representado por uma seta de entrada, os estados intermediários por círculos simples, e o estado de aceitação por um círculo duplo. As transições são rotuladas com os tokens consumidos em cada passo.

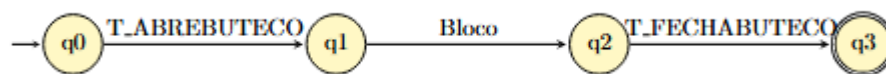


Figura 1 - Programa

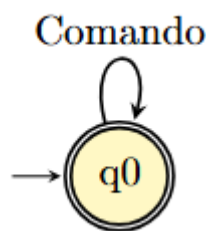


Figura 2 - Bloco

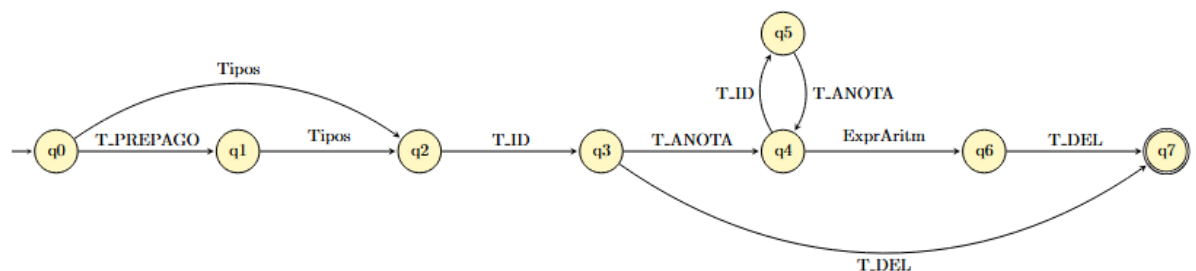


Figura 3 - DeclaraVar

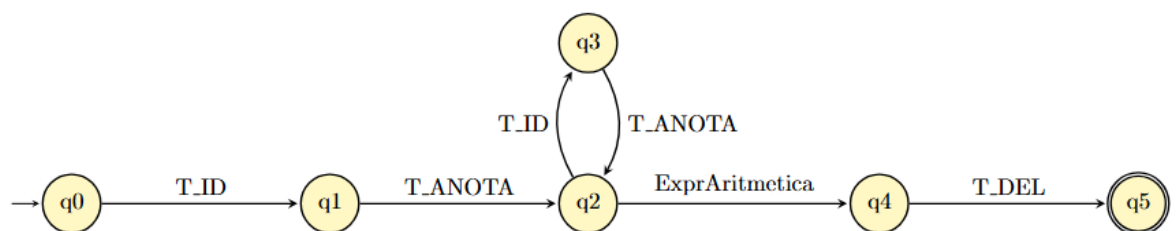


Figura 4 - Atribuicao

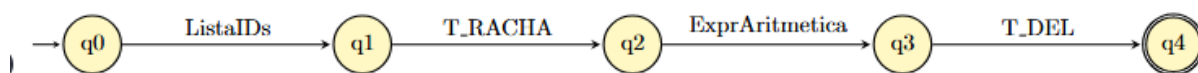


Figura 5 - Rachadinha

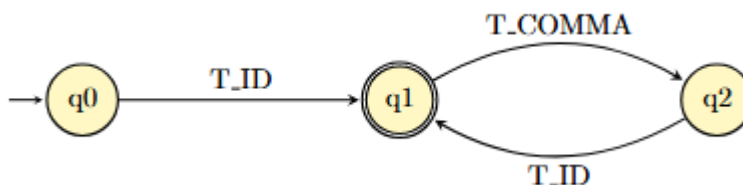


Figura 6 - ListalDs

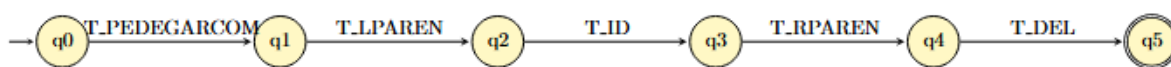


Figura 7 - Escuta



Figura 8 - Truco

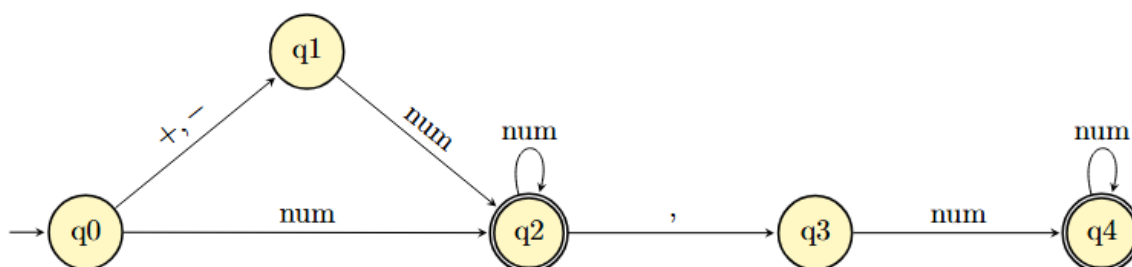


Figura 9 - T_NUM



Figura 10 - Condicional

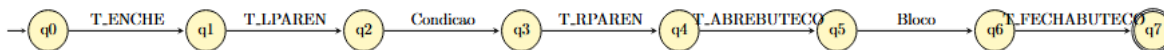


Figura 11 - Enquanto



Figura 12 - Para_Rodada

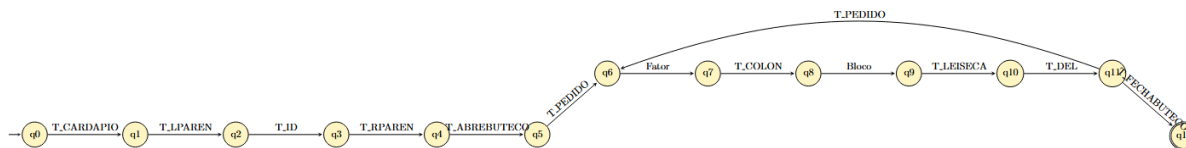


Figura 13 - Escolha_Cardapio

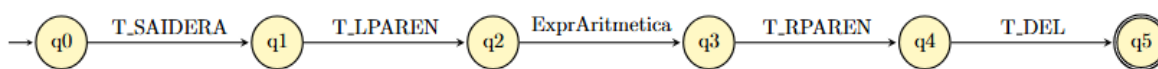


Figura 14 - Retorno

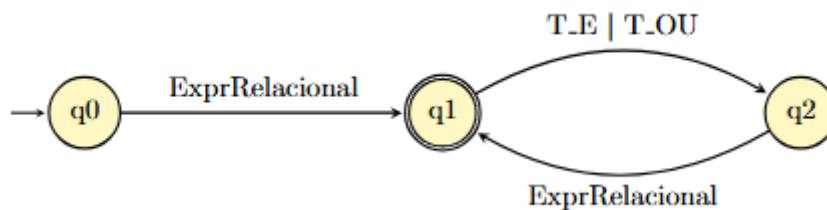


Figura 15 - Álgebra Booleana

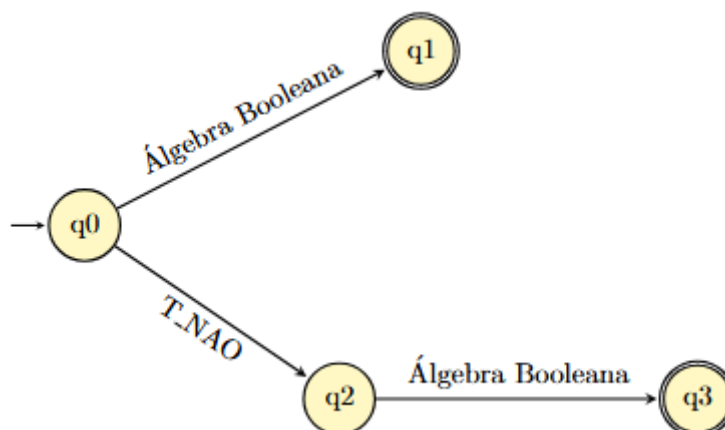


Figura 16 - Condição

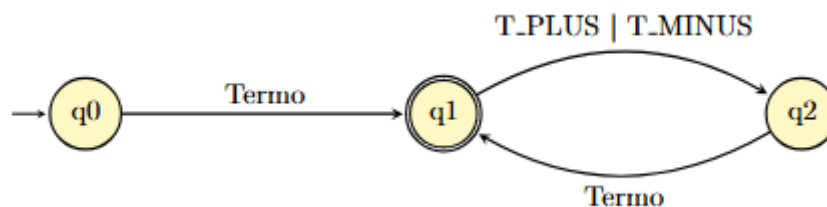


Figura 17 - ExprAritmética

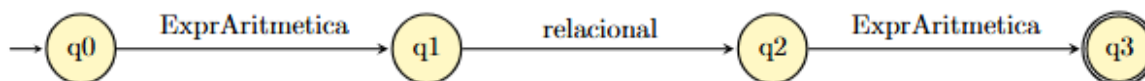


Figura 18 - ExprRelacional

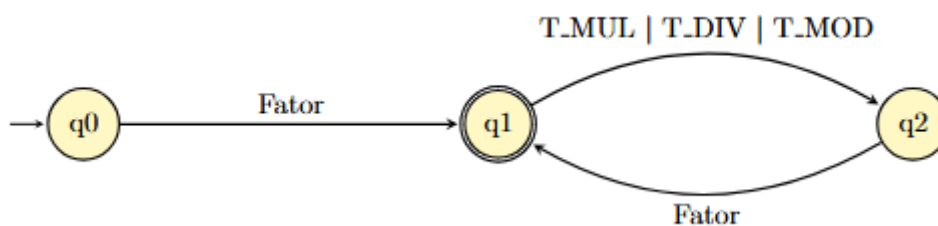


Figura 19 - Termo

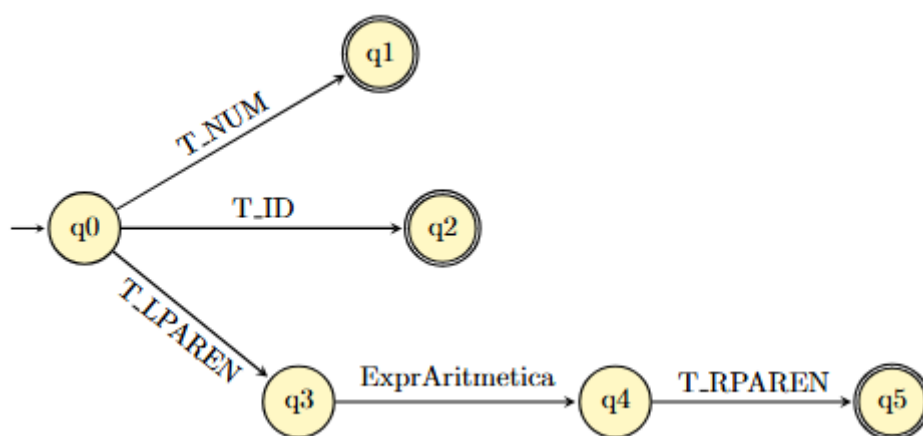


Figura 20 - Fator

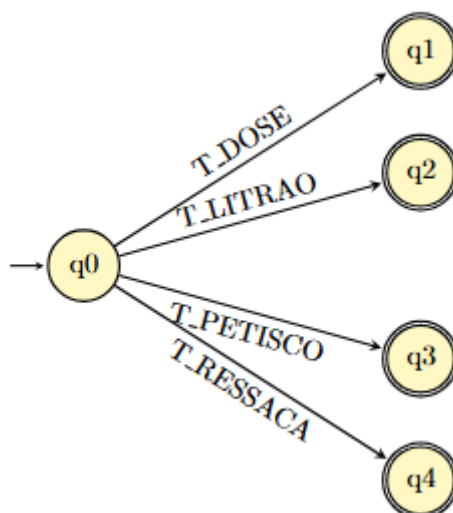


Figura 21 - Tipos

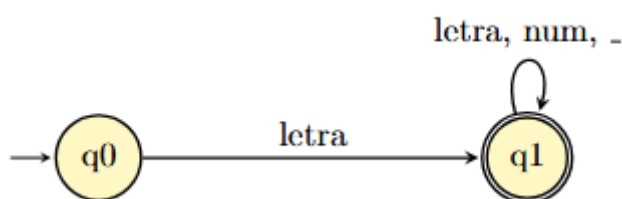


Figura 22 - T_ID

3 Conclusão

O trabalho demonstrou a aplicação prática dos conceitos fundamentais de compiladores na criação da linguagem de programação "buteCo". A partir da temática de bar, foi possível mapear estruturas clássicas da linguagem C para um vocabulário temático.

O projeto passou por todas as etapas de especificação de um compilador: a modelagem lexical com definição do vocabulário e tabela de tokens, a formalização através de expressões regulares, a construção de autômatos finitos determinísticos para a validação de cada construção sintática, e a elaboração de uma gramática livre de contexto não-ambígua com regras de produção.

Entre os diferenciais da linguagem, destacam-se o operador "racha" para atribuição distributiva, o modificador "pre_pago" como proteção semântica contra valores negativos e a inicialização implícita de variáveis com valor zero. Essas funcionalidades agregam valor prático à linguagem ao mesmo tempo em que mantêm a coerência temática.

Conclui-se que a teoria de linguagens formais e autômatos fornece a base matemática necessária para a construção de compiladores funcionais, e que a criatividade na definição do vocabulário léxico não compromete a capacidade de processamento formal da gramática, desde que os fundamentos teóricos sejam rigorosamente respeitados.